Chapter 6_1

MediaPipe 기반 제스처 인식

컴퓨터 비전 기술로 카메라 영상을 분석하고 사용자 행동을 인식하는 지능형 인터페이스 구현

컴퓨터 비전 | OpenCV | MediaPipe | 손 제스처 | 눈 깜빡임 인식

학습 목표

01
컴퓨터 비전 이해

이미지·영상 데이터 구조와 처리 흐름 파악

02
OpenCV 기초

실시간 카메라 입력 및 영상 출력 구현

03
MediaPipe 구조

Detection, Tracking, Landmark 개념 이해

04
눈 깜빡임 인식

Face Mesh로 임계값 기반 눈 상태 판별

컴퓨터 비전이란?

이미지·영상 데이터를 기반으로 객체를 인식하고, 특징을 추출하며, 패턴을 분석하는 인공지능 기술
→ 인간의 시각 인지 과정을 모방하여 컴퓨터가 시각 정보를 해석

처리 과정

① 영상 입력 — ② 객체 검출 — ③ 특징점 추출 — ④ 패턴 분석 — ⑤ 결과 생성

주요 기능

[1] 얼굴 검출 및 Facial Landmark 추출

[2] 손 랜드마크 기반 Gesture Recognition

[3] 눈꺼풀 분석을 통한 Eye Blink Detection

[4] 인체 관절 좌표 활용 Pose Estimation

이미지 데이터 구조

픽셀(Pixel)과 RGB

- 컴퓨터는 이미지를 수많은 픽셀의 집합으로 처리
- 각 픽셀 = (R, G, B) 세 채널로 구성된 벡터
- 각 채널은 0 ~ 255 사이의 정수 값
- 이미지 = 픽셀들의 2차원 배열 (높이 × 너비 × 3)

(255,0,0) 빨강	(0,255,0) 초록	(0,0,255) 파랑
(255,255,0) 노랑	(0,0,0) 검정	(255,255,255) 흰색

영상(Video) = 연속된 프레임

Frame 1 0ms — Frame 2 33ms — Frame 3 66ms — Frame 4 99ms

30 FPS = 1초에 30장 → 약 33ms 간격으로 처리

손 움직임 → 제스처 인식

눈 개폐 → 깜빡임 감지

신체 위치 → 자세 분석

OpenCV 기초

Open Source Computer Vision Library — 이미지 및 영상 처리를 위한 대표적인 라이브러리

카메라 입력

실시간 영상 캡처
VideoCapture

이미지 변환

BGR→RGB 변환
크기 조정·회전

객체 검출

윤곽선 검출
findContours

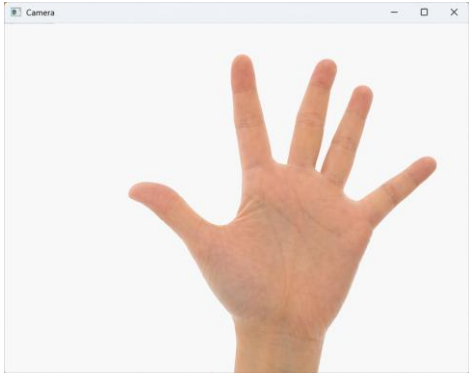
결과 시각화

텍스트·도형 출력
putText / imshow

기본 코드 구조

```
import cv2
cap = cv2.VideoCapture(0)
while cap.isOpened():
    success, frame = cap.read()
    if not success: continue
    cv2.imshow("Camera", frame)
    if cv2.waitKey(5) & 0xFF == 27: break
cap.release()
cv2.destroyAllWindows()
```

실행 결과



전체 코드 구조 한눈에 보기

```python
import cv2

cap = cv2.VideoCapture(0)

while cap.isOpened():
    success, frame = cap.read()
    if not success:
        continue

    cv2.imshow("Camera", frame)

    if cv2.waitKey(5) & 0xFF == 27:
        break

cap.release()
cv2.destroyAllWindows()
```

① 라이브러리 로드

② 카메라 초기화

③ 메인 루프 시작

④ 프레임 출력

⑤ 키 입력 감지 / 종료

⑥ 자원 해제

① import cv2 & ② VideoCapture(0)

```
import cv2
```

OpenCV 라이브러리	주요 기능 제공	import 의미
Open Source Computer Vision Library 영상 처리 분야 세계 표준 라이브러리	카메라 입력 제어 이미지 변환 및 필터링 객체 검출 및 결과 시각화	Python에 cv2 모듈을 메모리에 로드 이후 cv2.함수명() 으로 모든 기능 호출 가능

```
cap = cv2.VideoCapture(0)
```

0	기본 웹캠 (내장 카메라) 환경에 따라 1, 2로 변경 가능
초기화	카메라 드라이버와 연결 영상 입력 스트림 준비 완료
cap	카메라 장치 제어 객체 프레임 읽기 및 설정 관리
반환값	VideoCapture 객체 반환 cap.isOpened()로 성공 여부 확인

③ while cap.isOpened() & cap.read()

```
while cap.isOpened():
```

조건 확인

cap.isOpened() 는 카메라가 정상적으로 열려 있는지 확인
성공이면 True, 실패면 False 반환

반복 구조

True인 동안 내부 코드를 계속 반복 실행
실시간 영상 처리의 핵심 무한 루프 구조

안전 처리

카메라 연결이 끊기면 자동으로 루프 종료
비정상 종료 방지 역할 수행

```
success, frame = cap.read()
```

success

프레임 읽기 성공 여부

- True : 정상적으로 프레임을 읽음
- False : 카메라 오류 또는 스트림 끝

frame

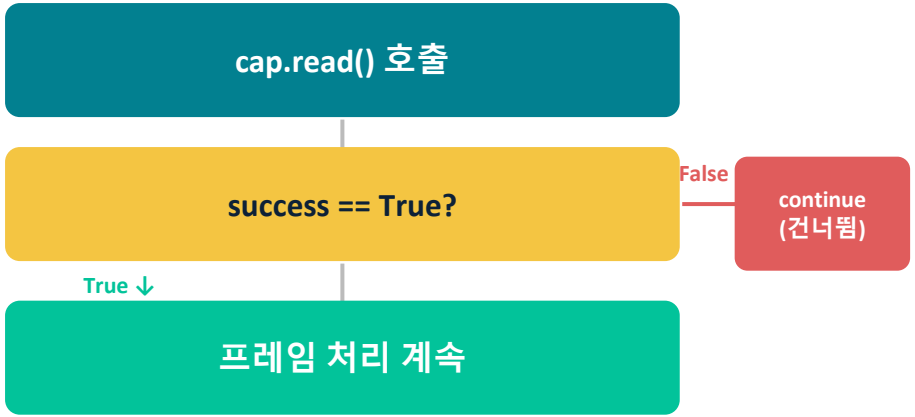
실제 영상 데이터

- NumPy 배열 형태의 이미지 데이터
- 높이 × 너비 × 3(RGB) 3차원 배열

④ if not success & continue

```
if not success:
    continue
```

처리 흐름



if not success

- success가 False일 때 실행
- 카메라 오류, 스트림 끊김 등
- 예외 상황을 조건문으로 처리

continue

- 현재 반복을 즉시 건너뜀
- while 루프 맨 위로 돌아가 다시 시작
- 프로그램 중단 없이 안정적으로 유지

왜 필요한가?

- 카메라가 순간 오류나도 프로그램 유지
- continue 없으면 None 프레임 처리 → 오류
- 실시간 시스템의 안전망(Safety Net) 역할

⑤ cv2.imshow("Camera", frame)

```
cv2.imshow("Camera", frame)
```

매개변수(Parameter) 분석

"Camera"

창 이름 (String)

- 화면에 표시될 창의 제목
- 동일한 이름 → 같은 창에 덮어쓰기
- 다른 이름 → 새 창 생성
- "Live Feed", "MediaPipe" 등 자유롭게 지정

frame

이미지 데이터 (ndarray)

- NumPy 배열 형태의 이미지
- cap.read()로 읽어온 한 장의 프레임
- 높이 × 너비 × 3 (BGR 채널) 배열
- 반복 실행으로 연속 출력 → 영상처럼 보임

핵심 imshow()가 while 루프 안에서 계속 호출되므로 → 매 프레임마다 새 이미지를 덮어씌워 **'영상'처럼 보이게 됩니다.**

⑥ waitKey(5) & 0xFF == 27 & break

```
if cv2.waitKey(5) & 0xFF == 27:
    break
```

waitKey(5)

- 5ms 동안 키보드 입력을 대기
- 입력된 키의 아스키 코드 반환
- 입력 없으면 -1 반환
- 영상 출력 속도(FPS) 제어 역할도 수행
- 값을 0으로 설정 시 무한 대기

& 0xFF == 27

- 비트마스킹(bitmasking) 연산
- & 0xFF : 하위 8비트만 추출
- → 64비트 OS 호환 처리
- 27 = ESC 키의 아스키(ASCII) 코드
- ESC 키 입력 여부를 확인하는 조건식

break

- while 루프를 즉시 탈출
- 이후 자원 해제 코드로 이동
- ESC 누르면 프로그램 안전 종료
- 없으면 강제 종료(X 버튼) 필요
- q키 종료: & 0xFF == ord('q')

⑦ cap.release() & destroyAllWindows()

```
cap.release()
cv2.destroyAllWindows()
```

cap.release()

- 사용 중인 카메라 장치(하드웨어)를 해제
- OS에 카메라 자원을 반환
- 호출하지 않으면 다른 프로그램이 카메라 사용 불가
- VideoCapture 객체와 연결된 버퍼 메모리 해제
- 반드시 while 루프 종료 후 호출해야 함

cv2.destroyAllWindows()

- OpenCV로 생성된 모든 창을 닫음
- imshow()로 열린 모든 윈도우 제거
- 특정 창만 닫으려면 destroyWindow('이름') 사용
- 호출 없이 종료 시 좀비 프로세스 발생 가능
- 프로그램 종료 시 항상 마지막에 호출

자원 해제 순서 (반드시 지킬 것)

break (루프 종료) — cap.release() — cv2.destroyAllWindows() — 프로그램 종료

전체 코드 완성본 & 흐름 요약

```python
import cv2

cap = cv2.VideoCapture(0)

while cap.isOpened():
    success, frame = cap.read()
    if not success: continue

    cv2.imshow("Camera", frame)

    if cv2.waitKey(5) & 0xFF == 27:
        break

cap.release()
cv2.destroyAllWindows()
```

①	라이브러리 로드	`import cv2`
②	카메라 초기화	`VideoCapture(0)`
③	루프 시작	`while cap.isOpened()`
④	프레임 읽기	`success, frame = cap.read()`
⑤	오류 처리	`if not success: continue`
⑥	화면 출력	`cv2.imshow("Camera", frame)`
⑦	키 입력 / 종료	`waitKey(5) & ESC → break`
⑧	자원 해제	`cap.release() + destroyAllWindows()`

MediaPipe란 무엇인가?

Google에서 개발한 **실시간 멀티미디어 처리 프레임워크**
이미지·영상에서 얼굴, 손, 신체 자세 등 다양한 객체를 빠르게 검출·분석하는 라이브러리

컴퓨터 비전 + ML 결합

머신러닝 모델과 컴퓨터 비전 기술을 통합하여 실시간 환경에서 고성능 제공

랜드마크 추출

단순 위치 검출을 넘어 객체 내 주요 특징점 좌표까지 추출 가능

다양한 응용 분야

손 제스처 인식
얼굴 특징 분석
자세 추정, 시선 추적

경량화 설계

실시간 인터랙션에 최적화
CPU/GPU 모두 지원
모바일 환경에서도 동작

MediaPipe 처리 과정

카메라 입력 → Detection 객체 위치 검출 → Tracking 객체 추적 → Landmark 특징점 추출 → 결과 출력

Detection vs Tracking

Detection (검출)

처음으로 객체를 찾아내는 과정

- 영상/이미지 안에서 특정 객체가 어디에 있는지 처음으로 탐색
- 화면 전체(넓은 영역)를 탐색
- 연산량이 많고 처리 속도 느림 → 무거운 처리
- 예: 카메라 화면에서 손이나 얼굴의 위치를 처음 찾는 과정
- Frame 1에서만 주로 수행

VS

Tracking (추적)

검출된 객체를 계속 따라가는 과정

- 한 번 검출된 객체를 이후 프레임에서 계속 추적
- 이전 프레임 위치 기반으로 좁은 범위만 탐색
- 연산량 대폭 감소, 처리 빠름 → 가벼운 처리
- 부드럽고 안정적인 인식 결과 제공에 중요한 역할
- Frame 2, 3, 4... 에서 수행

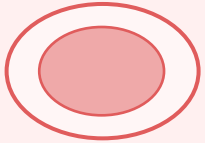
프레임별 처리 흐름 — 시간의 흐름에 따른 변화

시간 →

Detection

Frame 1

F1 · 0ms



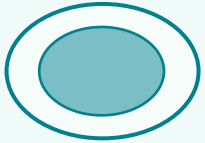
객체 첫 검출

전달

Tracking

Frame 2

F2 · 33ms



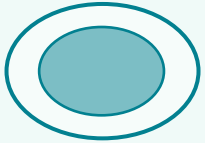
이전 위치 기반

전달

Tracking

Frame 3

F3 · 66ms



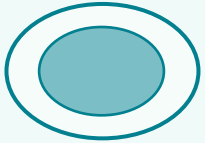
움직임 예측

전달

Tracking

Frame 4

F4 · 99ms



안정적 추적

Detection

"객체를 처음 찾는 단계"

- 전체 화면 탐색 → 연산량 많음
- 처음 1회(또는 객체 사라졌을 때) 수행
- 느리지만 정확한 위치 파악 가능

Tracking

"찾은 객체를 연속으로 따라가는 단계"

- 이전 위치 기반 → 좁은 범위만 탐색
- Frame 2부터 계속 반복 수행
- 빠르고 부드러운 안정적 추적 가능

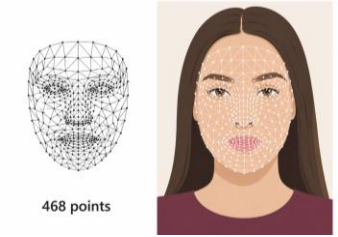
랜드마크(Landmark)란?

랜드마크 = 객체를 구성하는 주요 지점의 좌표
단순히 객체의 존재 여부만 파악하는 것이 아니라, 객체 내부의 중요한 특징점을 추출하여 보다 정밀한 분석 수행

얼굴 랜드마크

468개

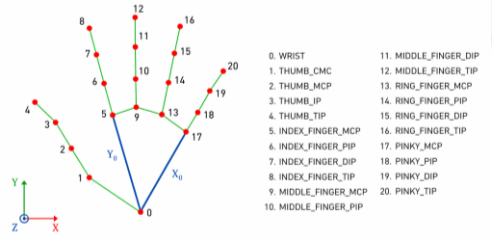
눈, 코, 입, 턱선 주변의 주요 지점
→ 눈 개폐 상태 판단
→ 표정 분석 및 시선 추적
→ 얼굴 방향 추정



손 랜드마크

21개

손목, 손가락 마디, 손가락 끝점 위치
→ 손가락이 펴졌는지 접혔는지 판단
→ 제스처 인식
→ 손 방향 추정



신체 랜드마크

33개

어깨, 팔꿈치, 무릎, 발목 등 관절 위치
→ 인체 자세 추정 (Pose Estimation)
→ 운동 자세 분석
→ 신체 비율 측정

손 랜드마크 21개 포인트 상세

랜드마크 번호 목록

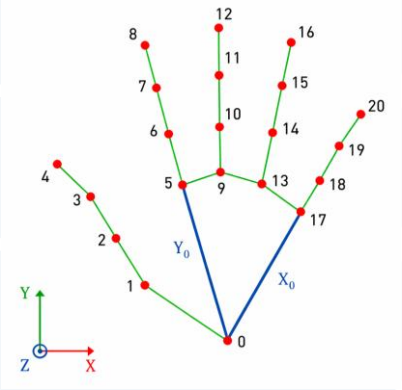
손목/엄지
0. WRIST
1. THUMB_CMC
2. THUMB_MCP
3. THUMB_IP
4. THUMB_TIP

검지
5. INDEX_FINGER_MCP
6. INDEX_FINGER_PIP
7. INDEX_FINGER_DIP
8. INDEX_FINGER_TIP

중지
9. MIDDLE_FINGER_MCP
10. MIDDLE_FINGER_PIP
11. MIDDLE_FINGER_DIP
12. MIDDLE_FINGER_TIP

약지
13. RING_FINGER_MCP
14. RING_FINGER_PIP
15. RING_FINGER_DIP
16. RING_FINGER_TIP

새끼
17. PINKY_MCP
18. PINKY_PIP
19. PINKY_DIP
20. PINKY_TIP



관절 이름 약어 설명

MCP
손가락 뿌리 관절
(손바닥과 손가락이 만나는 부분)

PIP
첫 번째 손가락 마디 관절
(손가락 중간 첫 번째)

DIP
두 번째 손가락 마디 관절
(손가락 끝 가까운 쪽)

TIP
각 손가락의 맨 끝 포인트
제스처 인식에 가장 많이 활용

CMC
엄지손가락 기저 관절
(손목과 엄지 사이)

랜드마크 활용 — 무엇을 알 수 있나?

랜드마크는 객체를 단순히 "찾는 것"을 넘어, 어떤 상태에 있는지 해석하는 핵심 정보

손 랜드마크 활용

손가락 굽힘/펼침

손가락 끝(TIP)과 두 번째 마디(PIP)의 y좌표 비교
→ TIP y > PIP y 이면 손가락이 접힌 상태

```
tip.y > pip.y  →  접힘
```

제스처 인식

여러 손가락의 굽힘 상태 조합으로 판별
→ 주먹, 가위바위보, 숫자 표현 등

```
count_fingers()  →  0~5
```

손 방향/위치

손목(0번)과 중지 MCP(9번) 좌표 비교
→ 손의 전체 위치와 기울기 계산 가능

```
wrist vs middle_mcp
```

얼굴 랜드마크 활용

눈 깜빡임 감지

눈 위(lm[159])와 아래(lm[145]) y좌표 거리
→ gap 값이 임계값 이하이면 눈 감김 상태

```
gap = lm[145].y - lm[159].y
```

표정 분석

입술 주변 랜드마크의 거리 변화
→ 웃음, 놀람, 찡그림 등 표정 분류

```
lip landmarks distance
```

고개 방향 추정

코 끝과 양쪽 귀 랜드마크 좌표 비교
→ 좌우/상하 고개 방향 계산

```
nose vs ears x/y
```

좌표 (x, y, z) 의 의미

MediaPipe는 각 랜드마크를 (x, y, z) 좌표 형태로 표현합니다.

x 가로 위치 (수평축)

0 (왼쪽) → 1 (오른쪽)

- 화면의 가로 방향 위치를 나타냄
- 값이 작을수록(0에 가까울수록) 화면 왼쪽
- 값이 클수록(1에 가까울수록) 화면 오른쪽
- 정규화된 값: 0.0 ~ 1.0 사이

x = 0.5 → 화면 정중앙 가로 위치

y 세로 위치 (수직축)

0 (위쪽) → 1 (아래쪽)

- 화면의 세로 방향 위치를 나타냄
- 값이 작을수록(0에 가까울수록) 화면 위쪽
- 값이 클수록(1에 가까울수록) 화면 아래쪽
- 컴퓨터 좌표계는 위→아래가 +방향

y = 0.3 → 화면 위쪽 1/3 지점

z 깊이 (카메라 거리)

값 작음(가까움) → 값 큼(멀음)

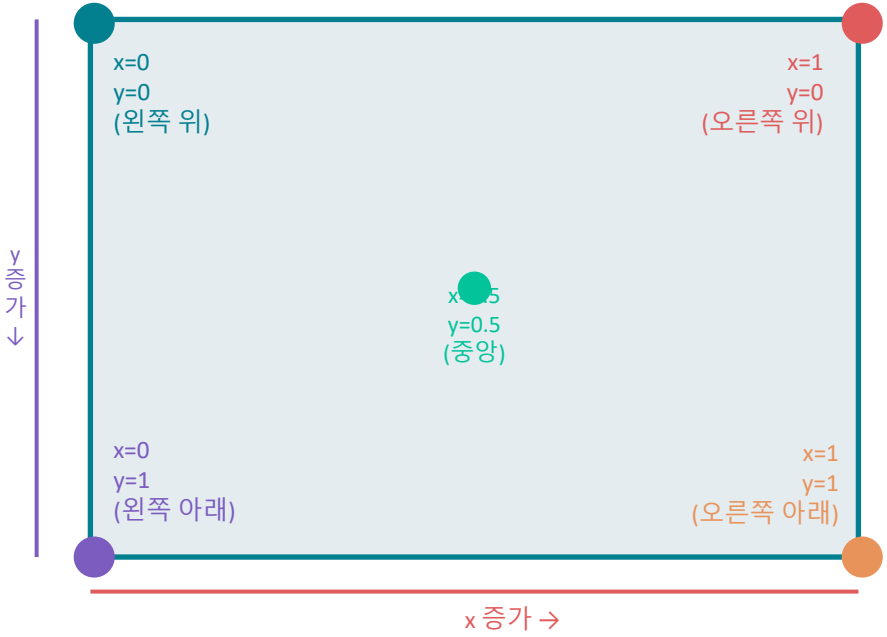
- 카메라를 기준으로 한 깊이(depth) 정보
- 객체가 카메라에 얼마나 가까운지 표현
- x, y와 달리 상대적인 값으로 해석
- 모델과 환경에 따라 기준이 달라짐

z 작음 → 카메라에 더 가까운 상태

정규화 좌표와 픽셀 좌표 변환

정규화(Normalization) = 화면 크기와 관계없이 0 ~ 1 사이의 비율 값으로 좌표를 표현
픽셀 단위의 절대 좌표가 아닌 상대적인 비율 값 → 서로 다른 해상도의 이미지에서도 동일하게 처리 가능

화면 좌표 범위



픽셀 좌표로 변환하기

```
# 정규화 좌표 → 픽셀 좌표 변환
h, w, _ = image.shape

x_pixel = landmark.x * w
y_pixel = landmark.y * h
```

1. image.shape로 영상의 높이(h)와 너비(w) 추출
2. landmark.x (0~1) × w = 실제 픽셀 x 위치
3. landmark.y (0~1) × h = 실제 픽셀 y 위치
4. 결과: 정수형으로 변환 후 cv2.circle() 등에 활용

Face Mesh 구조

MediaPipe Face Mesh = 얼굴의 주요 특징점을 정밀하게 추출하는 모델
약 468개의 랜드마크 기반으로 얼굴 형태 표현 — 눈, 코, 입, 턱선 주요 부위의 좌표를 함께 제공

깜빡임 감지	표정 분석	시선 추적	얼굴 인식
lm[159] vs lm[145] 눈 위아래 거리 비교	입·눈썹 움직임 랜드마크 변화 추적	홍채 랜드마크 위치 변화 분석	전체 특징점 비교 신원 확인 응용

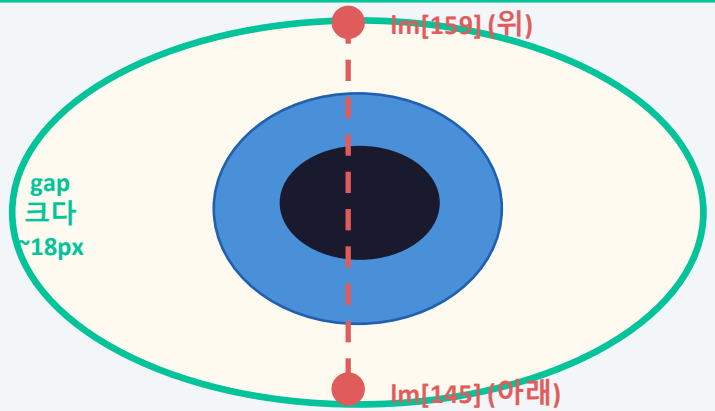
Face Mesh가 유용한 이유

- 단순 위치 검출을 넘어 얼굴 세부 특징점 좌표까지 제공
- 눈 개폐 상태, 입의 움직임, 얼굴 방향 등 다양한 정보 분석 가능
- 얼굴 인식, 표정 분석, 시선 추적, 깜빡임 감지 등 응용 분야 광범위

눈 좌표 분석 원리

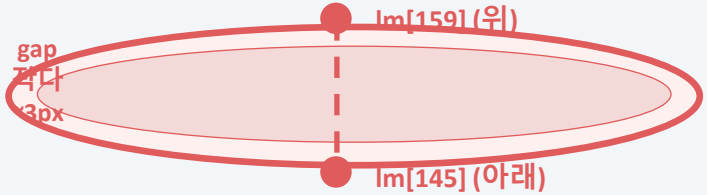
눈 주변 랜드마크의 좌표 변화 분석 → lm[159] (눈 위쪽)와 lm[145] (눈 아래쪽) 사이의 세로 거리(gap)를 계산
gap 값을 임계값(threshold)과 비교하면 눈의 열림과 닫힘 상태를 판단할 수 있습니다

눈 열림 (Open)



gap > threshold → OPEN

눈 닫힘 (Closed)



gap <= threshold → CLOSED

깜빡임 판별 알고리즘

1
랜드마크 추출
`lm[159], lm[145]`

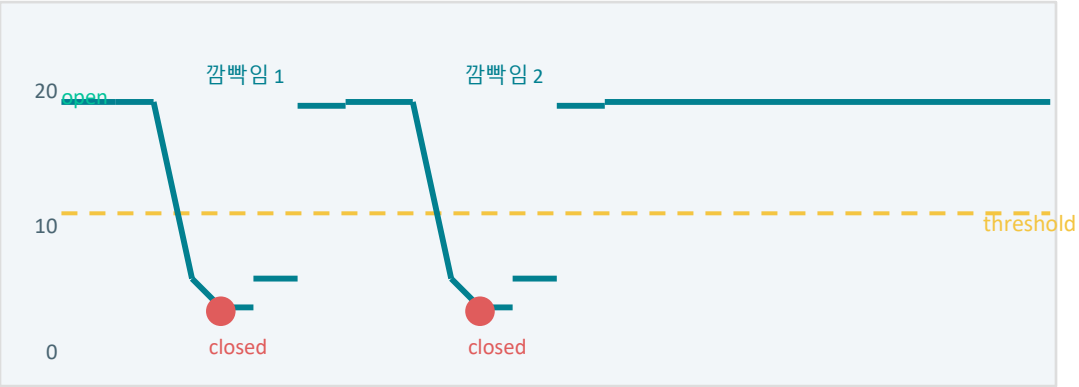
2
거리 계산
`gap = y145 - y159`

3
상태 변화 감지
`open → closed`

4
깜빡임 판별
`closed→open` 완료

깜빡임 1회 = open → closed → open 으로 변화하는 하나의 사이클
이전 상태(prev_state)를 저장 → closed에서 open으로 전환되는 시점에 깜빡임 횟수 +1 카운트

gap 값의 시간 변화 (프레임별)



1. 한 순간의 값이 아닌 패턴 분석 필요
2. prev_state 저장하여 상태 추적
3. closed → open 전환 시 카운트 +1
4. threshold는 환경에 맞게 조정 필요

전체 실습 코드 (12pt — 따라 쓰기용)

```python
import cv2
from mediapipe.python.solutions import face_mesh as mp_face_mesh

cap = cv2.VideoCapture(0)
with mp_face_mesh.FaceMesh(
    min_detection_confidence=0.5,
    min_tracking_confidence=0.5
) as face_mesh:

    while cap.isOpened():
        success, image = cap.read()
        if not success: continue

        image = cv2.flip(image, 1)
        rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
        results = face_mesh.process(rgb)
```

전체 실습 코드 (계속)

```
17
18        if results.multi_face_landmarks:
19            for face_landmarks in results.multi_face_landmarks:
20                lm = face_landmarks.landmark
21                h, w, _ = image.shape
22                top    = lm[159].y * h
23                bottom = lm[145].y * h
24                gap    = bottom - top
25
26                cv2.putText(image, f"gap: {gap:.1f}", (30, 80),
27                        cv2.FONT_HERSHEY_SIMPLEX, 2, (255, 255, 0), 4)
28
29                THRESHOLD = 10
30                if gap > THRESHOLD:
31                    cv2.putText(image, "OPEN", (30, 180),
32                    cv2.FONT_HERSHEY_SIMPLEX, 3, (0, 255, 0), 5)
```

전체 실습 코드 (완성)

```
33                  else:
34                      cv2.putText(image, "CLOSED", (30, 180),
35                      cv2.FONT_HERSHEY_SIMPLEX, 3, (0, 0, 255), 5)
36
37          cv2.imshow("Left Eye Test", image)
38          if cv2.waitKey(5) & 0xFF == 27:
39              break
40
41  cap.release()
42  cv2.destroyAllWindows()
```

실행 결과

```
gap: 9.5    CLOSED
```

```
gap: 14.7    OPEN
```

코드 설명 (1/3) — 라이브러리 & 초기화

```
import cv2
```

OpenCV 라이브러리 불러오기

- cv2 = Open Source Computer Vision의 약자
- 카메라 입력, 이미지 변환, 텍스트 출력, 창 표시 기능 제공
- 이후 cv2.함수명() 형태로 모든 기능 호출 가능

```
from mediapipe...import face_mesh as mp_face_mesh
```

MediaPipe Face Mesh 불러오기

- mediapipe 라이브러리에서 face_mesh 모듈만 불러옴
- as mp_face_mesh : 긴 이름을 짧게 별명으로 지정
- 얼굴 468개 랜드마크를 추출하는 기능 제공

```
cap = cv2.VideoCapture(0)
```

웹캠 장치 초기화

- 0 = 기본 내장 웹캠 (외부 카메라면 1, 2로 변경)
- cap 객체 = 카메라 제어 및 프레임 읽기 담당
- 이 한 줄로 카메라 드라이버와 연결 완료

```
mp_face_mesh.FaceMesh(min_detection_confidence=0.5, ...)
```

Face Mesh 모델 생성 (with문)

- min_detection_confidence=0.5 : 얼굴 첫 검출 최소 신뢰도 50%
- min_tracking_confidence=0.5 : 검출 후 추적 최소 신뢰도 50%
- with문 : 사용 후 자동으로 메모리 해제 (안전한 방식)

코드 설명 (2/3) — 영상 처리 & 랜드마크 추출

```
success, image = cap.read()
```

카메라에서 한 프레임 읽기

- success : 프레임 읽기 성공 여부 (True / False)
- image : 실제 영상 데이터 — NumPy 배열 (높이×너비×3)
- if not success: continue → 실패 시 그 프레임을 건너뜀

```
image = cv2.flip(image, 1)
```

화면 좌우 반전 (거울 효과)

- 1 = 수평(좌우) 반전 / 0 = 수직(상하) 반전
- 사용자가 거울처럼 자연스럽게 자신의 모습을 볼 수 있도록 설정
- 없으면 움직임 방향이 반대로 보여 어색함

```
rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

BGR → RGB 색상 변환

- OpenCV는 기본적으로 BGR 순서로 이미지를 읽음
- MediaPipe는 RGB 형식이 필요 → 반드시 변환 필요!
- 변환 없이 처리하면 색상 오류 및 인식 실패 발생

```
results = face_mesh.process(rgb)
```

얼굴 랜드마크 추출 실행

- rgb 이미지를 Face Mesh 모델에 입력
- results.multi_face_landmarks : 감지된 얼굴들의 랜드마크 목록
- 얼굴이 없으면 None 반환 → if 조건문으로 체크 필요

코드 설명 (3/3) — 눈 거리 계산 & 상태 판별

```
lm = face_landmarks.landmark  /  h, w, _ = image.shape
```

랜드마크 변수 저장 & 이미지 크기 추출

- lm : face_landmarks.landmark 를 짧게 저장 (편의상)
- image.shape → (높이 h, 너비 w, 채널 수) 반환
- 랜드마크 좌표는 0~1 정규화값 → h, w 곱하면 픽셀 좌표로 변환

```
top = lm[159].y * h  /  bottom = lm[145].y * h  /  gap = bottom - top
```

눈 위아래 거리(gap) 계산

- lm[159] = 왼쪽 눈의 위쪽 랜드마크 (y좌표 작음)
- lm[145] = 왼쪽 눈의 아래쪽 랜드마크 (y좌표 큼)
- gap = bottom - top → 눈이 열릴수록 값 커짐, 감길수록 값 작아짐

```
THRESHOLD = 10  /  if gap > THRESHOLD: → OPEN  /  else: → CLOSED
```

임계값 기반 눈 상태 판별

- THRESHOLD = 10 : 눈 열림/닫힘 기준값 (환경에 따라 조정 필요)
- gap > 10 → OPEN (초록색 텍스트) / gap <= 10 → CLOSED (빨간색)
- 먼저 gap 값을 화면에 출력하여 본인 환경의 최적값 확인 후 설정

```
cv2.putText(image, f'gap: {gap:.1f}', (30,80), ...)
```

화면에 텍스트 출력

- f'gap: {gap:.1f}' : f-string으로 gap 값을 소수점 1자리로 출력
- (30, 80) : 텍스트 출력 위치 (x=30픽셀, y=80픽셀)
- cv2.FONT_HERSHEY_SIMPLEX : OpenCV 기본 폰트 / 숫자는 크기·두께